8

Online DPO & Iterative Alignment

The Offline Limitation

Chapter 7 ended with a candid assessment of the weaknesses of DPO's. The most fundamental one is that DPO is *offline*: it trains on a fixed dataset of preference pairs and never generates new responses during training. The policy cannot discover strategies that are not already represented in the data.

This matters more than it might seem. Consider what happens as training progresses. The policy improves, which means that it is now a different model from the one that generated the preference data. The responses it would produce today are not the responses that were compared in the data set. It is learning from a world that it no longer inhabits.

This is the **distribution mismatch** problem. The preference data was collected under one distribution (the behavior of the SFT model or some earlier policy), but the model being trained has drifted to a different distribution. The further training progresses, the wider this gap grows and the less useful the original preference data becomes.

Distribution mismatch in practice

At the start of DPO training, the policy is close to the SFT model that generated the preference data. The log ratios $\log \frac{\pi_\theta}{\pi_{\text{ref}}}$ are small, the implicit rewards are well-calibrated, and learning proceeds smoothly.

After thousands of gradient steps, the policy has shifted. It now assigns very different probabilities to the responses in the dataset. Some "losers" in the original data might actually be good responses under the new policy's capabilities. Some "winners" might represent strategies the model has already surpassed. The preference labels are still correct, but they are no longer the most informative training signal available.

This is why vanilla DPO eventually plateaus: it exhausts the useful signal in its fixed dataset.

The solution is conceptually simple: generate new data from the current policy, collect new preferences on those data, and train again. This is the core idea behind iterative and online alignment methods.

Online vs Offline Preference Learning

Before diving into specific algorithms, it helps to clarify the distinction between online and offline approaches, because this is the axis along which the methods in this chapter differ from vanilla DPO.

Offline preference learning (vanilla DPO, Chapter 7) works with a fixed dataset $\mathcal{D} = \{(x_i, y_w^i, y_l^i)\}$ of prompts and preference pairs. The policy π_θ learns from $\mathcal{D}$ without generating any new responses during training. The dataset never changes.

Online preference learning has the policy π_θ generate new responses during training. These responses are scored (by a reward model, a judge model, or a verifier), new preference pairs are constructed, and the policy trains on this fresh data. The dataset grows or refreshes

at each iteration.

The cooking analogy

Offline is learning to cook by studying a fixed cookbook. You read the recipes, internalize what makes dishes good or bad, and develop taste. But you never actually cook. Your understanding is bounded by what the cookbook contains.

Online is learning to cook by actually cooking. You try a recipe, taste the result, adjust the seasoning, try again. Each iteration produces new data (a new dish to evaluate), and your learning is guided by your current skill level, not by recipes someone else wrote months ago.

The key tradeoff:

Dimension	Offline (DPO)	Online
Data	Fixed, collected once	Fresh, from current policy
Exploration	None	Policy discovers new strategies
Cost	Cheap (no generation)	Expensive (generation + scoring each iteration)
Stability	Very stable	Requires care to avoid instability
Ceiling	Bounded by data quality	Can exceed original data quality

There is also a distinction between **on-policy** and **off-policy** that is worth noting. On-policy methods (like PPO from Chapter 6) use responses generated by the current policy π_θ to update π_θ. Off-policy methods use responses generated by a different (usually older) policy. Online DPO is on-policy when it regenerates data each iteration, which is part of why it works well: the training signal always matches the model's current capabilities.

Online DPO: The Algorithm

Online DPO combines DPO's simplicity with RLHF's exploration. The idea is to run DPO iteratively, regenerating preference data from the current policy at each round.

The Online DPO Loop

For each iteration $k = 1, 2, \ldots, K$:

1. Generate. Sample a batch of prompts. For each prompt, use the current policy $\pi_\theta^{(k)}$ to generate multiple candidate responses.

2. Score. Evaluate the candidates using a judge. This could be a reward model, a stronger LLM acting as a judge, or an automated verifier (for math or code tasks).

3. Construct pairs. From the scored candidates, form preference pairs: higher-scoring responses become winners, lower-scoring responses become losers.

4. Train. Run one round of DPO on the fresh preference pairs, producing an updated policy $\pi_\theta^{(k+1)}$.

5. Update reference. Optionally update π_{ref} to the new policy before the next iteration (or keep the original SFT model as reference throughout).

Why this works. At each iteration, the preference data reflects the current policy's actual behavior. Winners are responses that the model *could* generate and that score well. Losers are responses the model *actually does* generate that score poorly. The training signal is always relevant to what the model is doing right now, which avoids the distribution mismatch that plagues vanilla DPO.

Comparison across methods:

	Vanilla DPO (Ch 7)	Online DPO	PPO (Ch 6)
Data source	Fixed dataset	Regenerated each iter	Generated each batch
Scoring	Human labels (offline)	Judge model or reward model	Reward model
Update rule	DPO loss	DPO loss	PPO clipped objective
Models in memory	2	2 (+ judge at scoring time)	4
Exploration	None	Per-iteration	Per-batch
Complexity	Low	Medium	High

Online DPO sits in a sweet spot: it recovers most of PPO's exploration benefit (the policy generates new data and learns from it) while keeping DPO's simpler training dynamics (no value function, no advantage estimation, no PPO clipping).

Practical Considerations

How often to regenerate data. The most aggressive approach regenerates preference data every training step, which is essentially equivalent to on-policy RL. The most conservative approach regenerates once every few epochs. In practice, regenerating every 1,000 to 5,000 training steps provides a good balance between freshness and compute cost.

How to generate preferences. There are three common approaches:

Reward model scoring. Generate N responses per prompt, score with a reward model, take the highest-scoring as winner and lowest as loser. This is cheap and fast but limited by the reward model's accuracy.

LLM-as-judge. Use a stronger model (such as GPT-4 or a large Claude model) to compare pairs of responses and declare a winner. This can capture nuances that a scalar reward model misses, but is slower and more expensive per comparison.

Automated verification. For tasks with objective correctness (math, code, factual questions), use a verifier: execute code and check test cases, verify mathematical proofs, or check answers against ground truth. This is the gold standard when available, because the signal is noise-free.

Synthetic preferences are often good enough. A common misconception is that preference data must come from human annotators. In practice, using a strong model as judge or using automated verifiers produces training data that is comparable in quality, at a fraction of the cost. The key advantage of Online DPO is not the source of preferences but the fact that preferences are collected on the current policy's outputs.

Whether to update the reference model. There are two strategies:

Fixed reference. Keep π_{ref} as the original SFT model throughout all iterations. The implicit KL penalty always measures drift from the starting point. This is safer but can become overly conservative as the policy improves across many iterations.

Rolling reference. Update $\pi_{\text{ref}} \leftarrow \pi_\theta^{(k)}$ at the start of each iteration. Each round of DPO measures drift from the *previous* iteration, not from the original. This allows larger cumulative changes but requires careful monitoring to prevent gradual drift into degenerate behavior.

Most practitioners start with a fixed reference and switch to a rolling reference only if they observe that the fixed reference is constraining improvement.

Iterative Self-Improvement: STaR

Online DPO uses external scoring (a judge or reward model) to construct preferences. But for tasks where correctness can be verified automatically, there is an even more elegant approach: let the model teach itself.

STaR (Self-Taught Reasoner) is an iterative self-improvement method designed specifically for reasoning tasks. Instead of preference pairs, STaR works with correct and incorrect solutions, using the model's own outputs as training data.

The STaR Loop

For each iteration:

1. Generate. The model attempts to solve a set of problems, producing step-by-step reasoning for each.

2. Verify. Check each solution against ground truth. Separate into correct solutions and incorrect solutions.

3. Rationalize failures. For problems the model got wrong, provide the correct answer and ask the model to generate reasoning that arrives at it. This produces "rationalized" reasoning traces.

4. Train. Fine-tune the model on all correct reasoning traces (both naturally correct and rationalized).

5. Repeat. The improved model can now solve harder problems, generating better training data for the next iteration.

The rationalization trick is the clever part. When the model fails a problem, we do not discard it. Instead, we tell the model the correct answer and ask: "How would you have gotten there?" The model generates a plausible reasoning chain that leads to the right answer. This is not as good as naturally correct reasoning (the model is working backwards from a known answer rather than discovering it), but it is far better than having no training data for that problem at all.

STaR Walkthrough

Problem: "If $2x + 5 = 17$, what is x?" **Correct answer:** $x = 6$

Scenario A: Model solves correctly

The model generates: "Let me solve step by step. $2x + 5 = 17$, so $2x = 17 - 5 = 12$, therefore $x = 12/2 = 6$."

Verification: $x = 6$ matches ground truth. This reasoning trace is stored as training data.

Scenario B: Model fails

The model generates: "Let me try: $2x = 17$, so $x = 8.5$." (It forgot to subtract 5.)

Verification: $x = 8.5$ is wrong.

Rationalization: We prompt the model with "The correct answer is $x = 6$. Generate step-by-step reasoning that arrives at this answer."

The model generates: "Given that $x = 6$: we start with $2x + 5 = 17$. Subtract 5 from both sides to get $2x = 12$. Divide by 2 to get $x = 6$."

This rationalized trace is also stored as training data.

Result: The model learns from both its successes and its failures. After training on these traces, it is less likely to forget the subtraction step in future problems.

Limitations of STaR

Verification dependency. STaR requires reliable ground truth or a verifier. This works well for math, code, and factual questions, but not for open-ended tasks like creative writing or style preferences. For those, use Online DPO instead.

Rationalization quality. Backward reasoning (given the answer, explain how to get there) is not the same as forward reasoning (discover the answer from scratch). Over-relying on rationalized traces can teach the model reasoning patterns that look correct but do not generalize. **Mitigation:** Mix rationalized and naturally correct traces, and always favor natural successes when available.

Compounding errors. If early iterations contain mistakes that pass verifi-

Expert Iteration

Expert Iteration takes a different approach to iterative improvement. Instead of using rationalization to learn from failures, it uses *expensive search* to produce high-quality training data, then *distills* that into the fast policy.

The intuition comes from game-playing AI. In chess, a policy network can suggest moves quickly, but a tree search that considers many possible futures can find much better moves given enough compute. Expert Iteration trains the fast policy to imitate the slow search, so the policy gradually absorbs the search's quality without needing the search at inference time.

The Expert Iteration Loop

For each iteration:

1. Generate with policy. Use the current policy π_θ to produce initial solutions for a batch of problems. This is fast but imperfect.

2. Improve with expert. For each problem, use an expensive search procedure to find better solutions. Typically: generate k candidates (e.g., $k = 20$) using beam search or sampling, score all candidates with a reward model or verifier, and select the best one.

3. Distill. Train π_θ on the (problem, expert_solution) pairs using standard supervised fine-tuning. The policy learns to produce expert-quality solutions directly, without needing the expensive search.

4. Repeat. The improved policy generates better initial solutions, which means the expert search starts from a better position, which produces even better training data.

The key insight is that search and learning are complementary. Search is expensive but high-quality. The policy is cheap but limited. By iterating between them, the policy gradually absorbs the search's quality. After several iterations, the policy alone (without search) can match the quality that previously required expensive search to achieve.

Expert Iteration for LLM Reasoning

Setup: The policy is a language model. The expert is best-of-N sampling with reward model scoring. The task is math problem solving.

One iteration in detail:

Step 1: Policy generates. Sample 1,000 math problems. For each, the policy generates one solution using standard decoding.

Step 2: Expert improves. For each problem, generate $N = 20$ candidate solutions using the policy with temperature sampling. Score all 20 with a reward model (or verify against ground truth). Select the highest-scoring solution as the "expert" solution.

Step 3: Distill. Fine-tune the policy on the 1,000 (problem, expert_solution) pairs.

Typical results over multiple iterations:

Iteration	Accuracy	Improvement
0 (baseline)	45%	–
1	58%	+13%
2	67%	+9%
3	72%	+5%
4	75%	+3% (diminishing returns)

Notice the diminishing returns. Each iteration, the policy absorbs more of the expert's quality, which means the gap between the policy and the expert shrinks. Eventually the policy is so good that the expert (best-of-N from the policy itself) cannot find significantly better solutions. At that point, you need either a better reward model, a larger N, or a more sophisticated search procedure to continue improving.

Expert Iteration vs Online DPO

Both are iterative methods that regenerate training data from the current policy. The difference is in the training signal:

Expert Iteration uses supervised learning: "Here is the best solution I found; learn to produce it directly." The policy imitates the expert's outputs.

Online DPO uses preference learning: "This solution is better than that

Choosing an Iterative Strategy

You now have three iterative approaches in your toolkit. Here is a framework for choosing among them.

Decision Framework: Task Type

Objective correctness (math, code, factual QA): Consider **STaR** (if you have ground truth answers) or **Expert Iteration** (if you have a verifier or strong reward model). Both leverage the fact that correct solutions can be identified automatically.

Subjective quality (helpfulness, style, safety): Use **Online DPO**. Pairwise preferences handle subjective judgments more naturally than binary correct/incorrect labels.

Mixed tasks: Use **Online DPO** as the general-purpose approach, supplemented with verifier-based signals where available.

Decision Framework: Scoring Mechanism

Ground truth answers: STaR is the most natural fit.

Reward model: Expert Iteration or Online DPO both work. Expert Iteration if you want supervised distillation; Online DPO if you want preference-based learning.

LLM-as-judge: Online DPO, since the judge naturally produces pairwise comparisons.

Decision Framework: Compute Budget

Tight budget: Vanilla DPO (Chapter 7) with a good static dataset may be sufficient. One round of Online DPO (generate once, train once) gives most of the benefit at modest extra cost.

Moderate budget: 3 to 5 iterations of Online DPO or Expert Iteration. This is the sweet spot for most practitioners.

Large budget: Many iterations with careful monitoring, or combine methods (e.g., Expert Iteration for reasoning, Online DPO for general helpfulness).

A practical starting recipe. If you are unsure where to begin: start with vanilla DPO on your initial preference data (Chapter 7). If performance plateaus and you have compute to spare, run one round of Online DPO by generating new responses from the trained model, scoring them, and training again. That single extra iteration often provides a meaningful boost at modest cost. If you need more, iterate further or switch to STaR/Expert Iteration for domains with verifiable correctness.

What we have built so far

SFT (Chapter 5): Teaches the model to follow instructions by imitating demonstrations. Bounded by demonstration quality.

RLHF with PPO (Chapter 6): Surpasses demonstrations using reward model guidance. Powerful but complex and expensive.

DPO (Chapter 7): Matches much of RLHF's benefit with half the models and a simpler training loop. But limited by its fixed offline dataset.

Online DPO and iterative methods (this chapter): Breaks through the offline ceiling by regenerating training data from the current policy. Recovers exploration benefits while keeping DPO's simplicity.

The next chapter covers reward modeling in greater depth (Chapter 9), because the quality of the scoring signal, whether it comes from a reward model, a judge, or a verifier, is the foundation that all of these iterative methods depend on. Then Chapter 10 surveys the full landscape of modern RL algorithms, including GRPO, RLOO, KTO, and others that make different tradeoffs than DPO and PPO.

Companion notebook. The code for this chapter is in `code/notebooks/part2_core/08_onli` in the book repository at `github.com/arunpshankar/packt-final`. On GitHub, navigate to the file and replace `github.com` with `githubtocolab.com` in the URL to open it directly in Colab.